

面向对象程序设计基础 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/html>

课程团队：刘知远 任炬 黄民烈

上期要点回顾

- 多文件编译和链接过程
- 宏定义、**Make**文件
- 程序命令行参数
- **GDB**调试工具

宏定义的使用

■ 防止头文件被重复包含

- 方法一 header guards

```
#ifndef __BODYDEF_H__  
#define __BODYDEF_H__  
// 头文件内容  
#endif
```

宏名

- 方法二 #pragma once

```
#pragma once  
// 头文件内容
```

■ 越来越多编译器支持 #pragma once

■ #pragma once 比 header guards 更简单，且保证物理上的同一文件不被编译/读取多次，更快

本讲内容提要

- 3.1 函数重载与缺省值
- 3.2 基础知识
- 3.3 类与对象
- 3.4 成员变量与成员函数
- 3.5 `private`和`public`
- 3.6 `this`指针
- 3.7 内联函数

为什么需要函数重载

■ 要表达“名一样而义不同”

- 同一任务，但输入信息的类型不同
 - `sum(int a, int b);`
 - `sum(double a, double b);`
- 同一任务，函数输入信息的存储形式不同
 - `sort(const char* s);`
 - `sort(std::string str);`
- 相似任务，在抽象概念层面一致
 - 如输出，有：显示到屏幕、打印到纸上、保存到文件等

第一节课：OOP的方法论：如何对客观世界进行思维抽象

函数重载

- 同一名称的函数，有两个以上不同的函数实现，被称为“函数重载”。如：

```
void print(const char* msg) { // <1>
    cout << "message: " << msg << endl;
}

void print(int score) { // <2>
    cout << "score = " << score << endl;
}
```

函数重载

- 编译器将根据函数调用语句的实际参数来决定哪一个函数被调用，如：

```
int main() {  
    print("Hello"); //调用print(const char*)  
    print(94); //调用print(int)  
    .....  
}
```

- 多个同名的函数实现之间，必须保证至少有一个函数参数的类型有区别。返回值、参数名称等不能作为区分标识。

为什么返回值不同不能作为区分

■ 假设：

```
float f(int s) {return s / 2.0;}
```

```
int f(int s) {return s * 2;}
```

■ 调用代码：

```
int main(){
```

```
    cout << f(3) << endl;
```

```
    // 编译器应该调用哪个函数呢?
```

```
    return 0;
```

```
}
```

内置类型转换

- 如果函数调用语句的实参与函数定义中的形参数据类型不同，且两种数据类型在C++中可以进行自动类型转换（如int和float），则实参会被转换为形参的类型，例如：

```
#include <iostream>
using namespace std;
void print(float score) {
    cout << "score = " << score << endl;
}
int main() {
    int a = 1;
    print(a); // 此时会将a转换为float型
    return 0;
}
```

内置类型转换

- 如果形参类型为**int**，实参类型为**float**，输出结果？

```
#include <iostream>
using namespace std;
void print(int score) { cout << "score = " << score << endl; }
int main() {
    print(1.0); // score = 1
    print(1.7); // score = 1
    print(2.3); // score = 2
    print(-3.9); // score = -3, 向零取整
    return 0;
}
```

- 自动类型转换也可以通过**定义的类型转换运算符**来完成（之后会讲）

内置类型转换

■ 以下程序的输出结果?

```
#include <iostream>
using namespace std;
void print(int score) { cout << "int = " << score << endl; }
void print(float score) { cout << "float = " << score << endl; }
int main() {
    float a = 1.0;
    print(a); // float = 1
    return 0;
}
```

- 当函数重载时，会优先调用类型匹配的函数实现，否则才会进行类型转换

函数参数的缺省值

- 函数参数可以在定义时**设置默认值（缺省值）**，
这样在调用该函数时，若不提供相应的实参，则
编译自动将相应形参设置成缺省值，如：

```
#include <iostream>
using namespace std;
void print(const char* msg = "hello") {
    cout << msg << '#';
}
int main() {
    cout << "Beijing...";
    print();
    return 0;
} // 输出 Beijing...hello#
```

函数参数的缺省值

- 有缺省值的函数参数，必须是**最后一个参数**

```
void print(char* name,  
           int score,  
           char* msg = "pass") {  
    cout << name << ": " << score  
        << ", " << msg << endl;  
}
```

- 如果有多个带缺省值的函数参数，则这些函数参数都只能在没有缺省值的参数后面出现，如：

```
void print(char* name, int score=0, char*  
msg="pass")
```

函数参数的缺省值

■ 缺省值的冲突问题

- 如果因为函数缺省值，导致了函数调用的**二义性**，编译器将拒绝代码。如下面代码，会导致编译不通过。

```
void fun(int a, int b=1) {  
    cout << a + b << endl;  
}
```

```
void fun(int a) {  
    cout << a << endl;  
}
```

//测试代码

fun(2); //编译器不知道该调用第一个还是第二个函数

auto关键字

■ C++11语法，需要std=c++11编译

■ 由编译器根据上下文自动确定变量的类型，如

`auto i = 3;` //i是int型变量

`auto f = 4.0f;` //f是float型变量

`auto a('c');` //a是char型变量

`auto b = a;` //b是char型变量

`auto *x = new auto(3);` //x是int*

auto关键字

■ 追踪返回类型的函数

- 可以将函数返回类型的声明信息放到函数参数列表的后面进行声明，如：

- 普通函数声明形式

```
int func(char* ptr, int val);
```

- 追踪返回类型的函数声明形式

```
auto func(char* ptr, int val) -> int;
```

- 追踪返回类型在原本函数返回值的位置使用auto关键字

auto关键字

■ **auto** 变量必须在编译期确定其类型

■ **auto** 变量必须在定义时初始化

- `auto a;` **// 错误**
- `auto b4 = 10, b5 = 20.0, b6 = 'a';`
// 错误, 没有推导为同一类型

■ 参数不能被声明为 **auto**

- `void func(auto a) {...}` **// 错误**

■ **auto** 并不是一个真正的类型。不能使用一些以类型为操作数的操作符，如 **sizeof** 或者 **typeid**。

- `cout << sizeof(auto) << endl;` **// 错误**

decltype

■ decltype

- decltype可以对变量或表达式结果的类型进行推导
- 重用匿名类型

```
struct { int d ;  
        double b;  
} anon_s; //没有名字的结构体，定义了一个变量  
int main() {  
    decltype(anon_s) as ;  
    //定义了一个上面匿名的结构体...  
}
```

decltype

■ decltype

- decltype可以对变量或表达式结果的类型进行推导。

```
struct { char name[17]; } anon_u;
```

```
struct {
```

```
    int d;
```

```
    decltype(anon_u) id;
```

```
} anon_s[100]; // 匿名的struct数组
```

```
int main() {
```

```
    decltype(anon_s) as; // 注意变量as的类型:数组
```

```
    cin >> as[0].id.name;
```

```
    ...
```

```
}
```

auto+decltype

■ 结合auto和decltype，自动追踪返回类型

- 可以推导返回类型（C++11）

```
auto func(int x, int y) -> decltype(x+y)
{
    return x+y;
}
```

- C++14中不再需要显式指定返回类型

```
auto func(int x, int y)
{
    return x+y;
}
```

WHY auto?

- 用于代替冗长复杂、变量使用范围专一的变量声明。

```
std::vector<std::string> vs;  
for (std::vector<std::string>::iterator i = vs.begin(); i !=  
vs.end(); i++)  
{ //..... }
```

```
std::vector<std::string> vs;  
for (auto i = vs.begin(); i != vs.end(); i++)  
{ //..... }
```

WHY auto?

- 在定义模板函数时，用于声明依赖模板参数的变量类型。

```
template <typename _Tx, typename _Ty>
void Multiply(_Tx x, _Ty y)
{
    auto v = x*y; //临时变量
    std::cout << v;
}
```

//使用时

Multiply(2, 3); //Multiply(int, int)

Multiply(2, 3.3); //Multiply(int, double)

WHY auto?

- 结合**auto**和**decltype**，自动追踪返回类型

```
template <typename _Tx, typename _Ty>
```

```
auto multiply(_Tx x, _Ty y)->decltype(x*y)
```

```
//C++11语法，C++14可省略"->"和decltype
```

```
{
```

```
    return x*y;
```

```
}
```

```
//使用时
```

```
auto a = multiply(2, 3.3); //a=6.6
```

内存申请与释放

■ 内存的动态申请与释放

- 指针变量所指内存可以通过new/delete运算符在程序运行时动态生成和删除，如：

`int * ptr = new int(10);` // 单个变量

`int * array = new int[10];` // 10元素数组

`delete ptr;` // 删除指针变量所指单个内存单元

`delete[] array;` // 删除多个单元组成的内存块

零指针

■ 0、NULL、nullptr

- 在C++中，NULL被定义为0

```
#ifdef __cplusplus  
#define NULL 0  
#else  
#define NULL ((void *)0)  
#endif
```

- 在C++11之前，可以使用NULL或者0表示空指针。
- 这么做有什么问题？

零指针

■ 减少NULL的使用 (c++11之前)

- 定义一个函数，并对它进行调用

```
void f(int x, int y) {...}
```

```
f(2, 0);
```

- 如果我们想对这个函数进行重载，并传入一个空指针作为参数

```
void f(int x, double *y) {...}
```

```
f(2, NULL);☹ // 实际调用的不是我们所期望的
```

```
f(2, static_cast<double *>(0));☺
```

- 当我们使用NULL表示空指针时，容易忽略它同时是一个int型常量。

零指针

■ `nullptr` 的引入 (C++11)

- `nullptr` 表示严格意义上的空指针。
- 此时再执行之前的代码，不会产生错误。

```
void f(int x, int y) {  
    cout<<"int"<<endl;  
}  
  
void f(int x, double *y) {  
    cout<<"pointer"<<endl;  
}  
  
f(2,nullptr);
```

Output: pointer

For循环

■ 基于范围的for循环

- 在循环头的圆括号中，由冒号":"分为两部分，第一部分是用于迭代的变量，第二部分则表示将被迭代的范围。

如：

```
#include <iostream>
using namespace std;
int main() {
    int arr[3] = {1, 3, 9};
    for (int e : arr) // auto e:arr 也可以
        cout << e << endl;
    return 0;
}
```

OOP从认识“对象”开始.....

■ 对象，是对现实世界中实际存在事物的抽象描述，它可以是有形的，也可以是无形的

- (1) 对象具有自己的静态特征和动态特征
 - 静态特征 —— 可以用某种数据来描述的属性；
 - 动态特征 —— 对象表现的行为或具有的功能。
- (2) 对象是由一组属性数据和对这些数据进行特定操作的一组服务所构成的“结合体”（概念）。

■ 封装 = {属性/数据，服务/函数}

封装的“装”——数据抽象

■ 数据+函数——从设计思想上看

- 封装（数据+函数）是OOP的基本特征
- 从只关心数值的存储与表示，到既考虑内容，又考虑语义（数据支持的计算或操作），形成了对“数据”概念的更本质认识，这种思维过程称为“数据抽象”。

用户定义类型——类class

■ class 用户自定义的类型

- 包含函数与数据的特殊“结构体”，用于扩充C++语言的类型体系
- 类中包含的函数，称为“成员函数”
- 包含的数据，称为“成员变量”

■ 成员函数必须在类内声明，但定义（实现）可以在类内或者类外。

■ 类=“属性/数据”+“服务/函数”

在头文件中声明类class

```
// agent.h
```

```
#ifndef AGENT_H
```

```
#define AGENT_H
```

```
class Agent {
```

```
    std::string memory[6];           // 成员变量
```

```
public:
```

```
    void plan(std::string goal);     // 成员函数（声明）
```

```
};
```

```
#endif
```

在实现文件中定义成员函数

```
// agent.cpp
#include "agent.h"
// 类外需要类名限定
void Agent::plan(std::string goal)
{
    ... // 函数实现
}
```

- 通常，类的**声明**放在头文件中，而类的成员函数**实现**（也叫**定义**）则放在实现文件中。
- 为了便于管理和代码复用，一般是将不同的类分别保存为不同的头文件和实现文件。

成员函数的两种定义方式

```
class Agent {  
public:  
    void plan(std::string goal) {  
        ...; //在类内定义成员函数  
    }  
    ...  
}; // <1>
```

```
void Agent::plan(std::string goal) {  
    ...; //在类外定义成员函数  
} // <2>
```

- 为了方便解决依赖关系，复杂的成员函数声明和定义一般是分离的，很少使用类内定义的方式。

类成员的访问权限

- 类的成员（数据、函数）可以根据需要分成组，不同组设置不同的访问权限

- **public**

- 被public修饰的成员可以在类外访问。

- **private**

- 默认权限
 - 被private修饰的成员不允许在类外访问。

- **protected**

- 以后介绍。

用户定义类型——类class

- 定义类后，可以像语言内建的类型一样，用类来定义变量，该变量通常被称为“对象”
- 通过“对象名.成员名”的形式，可以使用对象的数据成员，或调用对象的成员函数。在类外使用时仅限于访问public权限的成员
- 同样，可以使用“对象指针->成员名”形式访问数据成员或成员函数。在类外也只限于访问public权限的成员。

类成员的访问权限

```
// agent.h
```

```
class Agent {
```

```
public:
```

```
    void plan(std::string goal);
```

```
private:
```

```
    std::string memory[6];
```

```
}; // <1>
```

```
// 或者
```

```
class Agent {
```

```
    std::string memory[6]; // 成员的缺省属性为private
```

```
public:
```

```
    void plan(std::string goal);
```

```
}; // <2>
```

类成员的访问权限

```
// agent.cpp
#include "agent.h"           // Agent类的声明
int main()
{
    Agent obj;               // 定义变量（对象）
    obj.plan("Complete this code ..."); // 访问公有成员
    obj.memory[0] = "You are ..."; // ERROR!
    return 0;
}
```

- 不允许在类外(非该类的成员函数)操作访问对象的私有成员和保护成员，只能访问它的公有属性的成员（函数、数据）。

类成员的访问权限

```
// agent.h
class Agent {
private:
    std::string memory[6];
    void copy(Agent a);
public:
    void plan(std::string goal);
};
```

```
// agent.cpp
#include "agent.h"

void Agent::copy(Agent a) {
    for(int i=0; i<6; i++){
        memory[i] = a.memory[i];
        // 可以在类内用“.”操作访问同一类
        // 下的私有成员
    }
}
```

this指针

- 所有成员函数的参数中，**隐含**着一个指向当前对象的指针变量，其名称为**this**

```
class Agent {  
public:
```

这也是成员函数与普通函数的重要区别

```
    void plan(std::string goal) { // <1> 在类内定义成员函数
```

```
        ...
```

```
        this->memory[1] = goal; // 等价于 data[0][0] = 1;
```

```
    }
```

```
    ...
```

```
};
```

```
void Agent::plan(std::string goal) { // <2> 在类外定义成员函数
```

```
    this->memory[1] = goal; // 等价于 data[0][0] = 1;
```

```
    ... ;
```

```
}
```

思考

- 基本类型和自定义类型的差别是什么?
- 基本类型: **int, long, char, double, float**
 - 数据是什么, 操作是什么?
- 自定义类型
 - 数据是什么? 操作是什么?

```
class Test {  
    int data[100];  
public:  
    void setdata(const int*);  
    const int* getdata();  
    void operation1(int);  
};
```

内联函数

■ 为什么用内联函数

- 考虑一个很常用的函数

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

- 函数调用要进行一系列准备和后处理工作(压栈、跳转、退栈、返回等)，所以函数调用是一个比较慢的过程。
- 如果大量调用**max**函数，会拖慢程序。

内联函数

■ 为什么用内联函数

- 考虑一个很常用的函数

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

- 比较下面两种实现方式，函数比等价的表达式要慢得多！

```
cout << max(a, b) << endl;  
cout << (a > b ? a : b) << endl;
```

内联函数

■ 为什么用内联函数

- 使用内联函数，编译器自动产生等价的表达式。

```
inline int max(int a, int b) {  
    return a > b ? a : b;  
}  
cout << max(a, b) << endl;
```

- 上述代码等价于

```
cout << (a > b ? a : b) << endl;
```

内联函数

■ 内联函数和宏定义的区别

- 宏定义

```
#define MAX(a, b) (a) > (b) ? (a) : (b)
```

```
cout << MAX(a, b) << endl;
```

- 编译器会将所有宏定义的代码，直接拷贝到被调用的地方。上面对于**MAX**的调用，经过编译预处理器后，和下面代码完全等价。

```
#define MAX(a, b) (a) > (b) ? (a) : (b)
```

```
cout << (a) > (b) ? (a) : (b) << end;
```

内联函数

■ 内联函数和宏定义的区别

- 宏代码容易出错，编译预处理器在拷贝代码时，可能产生意想不到的边界效应。

```
#define MAX(a, b) (a) > (b) ? (a) : (b)
```

```
cout << MAX(a, b) + 2 << endl;
```

```
cout << (a) > (b) ? (a) : (b) + 2 << end; //编译预处理器结果☹
```

- 上述代码更改如下可正常工作。

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

```
cout << MAX(a, b) + 2 << endl;
```

```
cout << ((a) > (b) ? (a) : (b)) + 2 << end; // 编译预处理器结果☺
```

- 这样的宏代码万无一失了吗？

内联函数

■ 内联函数和宏定义的区别

- 显然不是万无一失的，例如下面的这种情况。

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

```
cout << MAX(a++, b) + 2 << endl;
```

```
cout << ((a++) > (b) ? (a++) : (b)) + 2 << end; // a被两次求值☹
```

- 因为宏代码是直接拷贝到指定位置的，很多缺陷不可避免。
- 内联函数的好处显而易见！

内联函数

■ 内联函数和宏定义的区别

- 内联函数可以执行类型检查，进行编译期错误检查。
- 内联函数可调试，而宏定义的函数不可调试。
 - 在**Debug**版本，内联函数没有真正内联，而是和一般函数一样，因此在该阶段可以被调试。
 - 在**Release**版本，内联函数实现了真正的内联，增加执行效率。
- 宏定义的函数无法操作私有数据成员。
- 宏使用的最常见场景：**字符串定义、字符串拼接、标志粘贴。**
- **再次强调**
 - 宏定义只是拷贝代码到被调用的地方。
 - 内联函数生成的是，和函数等价的表达式。

内联函数

■ 内联函数的注意事项

- 避免对大段代码使用内联修饰符。
 - 内联修饰符相当于把该函数在所有被调用的地方拷贝了一份，所以大段代码的内联修饰会增加负担。（代码膨胀过大）
- 避免对包含循环或者复杂控制结构的函数使用内联定义。
 - 因为内联函数优化的，只是在函数调用的时候，会产生的压栈、跳转、退栈和返回等操作。所以如果函数内部执行代码的时间比函数调用的时间长得多，优化几乎可以忽略。
- 避免将内联函数的声明和定义分开
 - 编译器编译时需要得到内联函数的实现，因此多文件编译时内联函数先需要将实现写在头文件中，否则无法实现内联效果。
- 定义在类声明中的函数默认为内联函数。
- 一般构造函数、析构函数都被定义为内联函数。

内联函数

//test.h

class Test {

int* data;

public:

void setdata(int* d) {data = d; } //内联函数

const int* getdata() {return this->data;} //内联函数

void operation1(int);

Test(int i){

if (i>0)

data = new int[i];

else

data = nullptr;

} //内联函数

~Test(){delete[] data;} //内联函数

};

内联函数

■ 内联函数的注意事项

- 内联修饰符更像是**建议**而不是命令。
- 编译器“**有权**”**拒绝不合理**的请求，例如编译器认为某个函数不值得内联，就会忽略内联修饰符。
- 编译器会对一些**没有内联修饰符**的函数，**自行判断可否转化为内联函数**，一般会选择短小的函数。

课后阅读

■ 《C++编程思想》（网络学堂可以下载pdf）

- 第5章，隐藏实现
- 第7章，函数重载与默认参数
- 第9章，内联函数

■ 如何在C++中打印变量类型

- <https://stackoverflow.com/questions/81870/is-it-possible-to-print-a-variables-type-in-standard-c>

```
#include<iostream>
#include<typeinfo>
#include<cxxabi.h>

class Test{

};

template<typename T>
char* get_type(const T& instance){
    return abi::__cxa_demangle(typeid((instance)).name(), nullptr, nullptr, nullptr);
}

int main(){
    auto* x = "abc";
    auto y = new int[5];
    auto z = Test();
    std::cout << get_type(x) << std::endl;
    std::cout << get_type(y) << std::endl;
    std::cout << get_type(z) << std::endl;
}
```

运行结果:
char const*
int*
Test

课后练习（无需提交）

■ 下面是一个类的声明

- 请提供成员函数的实现（注意实现和定义分开）
- 并测试这个类是否正常运行（你还需要测试访问权限）

```
class Move {  
    private:  
        double x;  
        double y;  
        double calc(); // calculate x * y  
    public:  
        Move(double a = 0, double b = 0);  
            // init x and y to a and b  
        void display(); // display the result of calc()  
        double get_x(); // return x  
        double get_y(); // return y  
        void reset(double a = 0, double b = 0);  
            // reset x and y to a and b  
};
```

课后练习（无需提交）

■ 函数重载、参数默认值

- 编写一个接受一个字符串参数，并打印该字符串的函数。不过，如果提供了第二个参数（bool 类型），且该参数为 true，则该函数打印 n 次字符串，其中 n 是该函数被调用的次数。

输出：

```
int main() {  
    f("OOP0");  
    f("OOP1");  
    f("OOP2", true);  
    f("OOP3", false);  
    f("OOP4", true);  
    return 0;  
}
```

OOP0
OOP1
OOP2
OOP2
OOP2
OOP3
OOP4
OOP4
OOP4
OOP4
OOP4

课后练习（无需提交）

■ 内联函数

- 创建两个功能相同的函数 `f1()` `f2()`。`f1()` 是内联函数，`f2()` 是非内联函数。使用 `<ctime>` 中的标准 C 函数 `clock()` 标记这两个函数的开始点和结束点，比较它们哪一个运行的更快，为了得到有效数字，也许需要在计时循环中重复调用这两个函数。
- 注意：由于编译器可能会忽略 `inline` 或自动执行其他优化。该特性和编译器优化指令有关，如 `"-O2"`。

结束